Deep Learning

Sungyoon Lee

Department of Computer Science
Hanyang University

sungyoonlee@hanyang.ac.kr

Understanding Language

- NNs for Language
- Transformer
- LLM

Lecture 15

Understanding Language

Understanding Language
NNs for Language
Transformer
LLM

Natural Language Processing (NLP)

- text classification
  - email spam classification
  - sentiment analysis
- machine translation
- speech recognition
- image captioning
- speech tagging
- document summarization
- question answering
- ...

Various Issues in Text Data

- documents have a variable length (unstructured)
  $\rightarrow$ TF-IDF $\in \mathbb{R}^{V \times N}$ (V: # vocab., N: # documents)
  - words are categorical variables with many possible values ($V \gg 1$)
  - one-hot encoding[1] has no natural notion of similarity
    $\rightarrow$ word embedding $\mathbb{R}^V \rightarrow \mathbb{R}^K$ ($K < V$)
- may encounter words at test time that have not been seen during training (so-called out-of-vocabulary or OOV words)
  $\rightarrow$ replace all novel words with the special symbol UNK, which stands for "unknown".[2]

[1] categorical variable to one-hot encoding, e.g., one-hot(red)$= [1, 0, 0]$, one-hot(green)$= [0, 1, 0]$, one-hot(blue)$= [0, 0, 1]$

[2] We could work at the character level. ... subword units, wordpieces, byte-pair encoding

bag of words, term frequency (TF) matrix, document frequency (DF), inverse DF, TF-IDF

For the vocabulary $\{v_i\}_{i=1}^{V}$ and the documents $\{x_n\}_{n=1}^{N}$, we have

$$\mathrm{BOW}_i(x_*)^{3} := \sum_{t=1}^{|x_*|} \mathbf{1}(x_{*t} = v_i) \qquad \text{(how many } v_i\text{'s in } x_*\text{)}$$

$$\mathrm{TF}_{in}{}^{4} := \sum_{t=1}^{|x_n|} \mathbf{1}(x_{nt} = v_i) \qquad \text{(how many } v_i\text{'s in } x_n\text{)}$$

$$\mathrm{DF}_i{}^{5} := \sum_{n=1}^{N} \mathbf{1}(v_i \in x_n) \qquad \text{(how many doc.s have } v_i\text{)}$$

$$\mathrm{IDF}_i := \log\left(\frac{N}{1+\mathrm{DF}_i}\right)^{6}$$

$$\mathrm{TFIDF}_{in} := \log(\mathrm{TF}_{in}+1) \times \mathrm{IDF}_i$$

[3] BOW$\in \mathbb{R}^V$ (as a structured data); vector space model

[4] $\mathrm{TF} = [\mathrm{BOW}(x_1)\ \mathrm{BOW}(x_2)\ \cdots\ \mathrm{BOW}(x_N)] \in \mathbb{R}^{V\times N}$

[5] DF = the sum of the number of the nonzero elements in each row of TF

[6] to reduce the impact of words that occurs many times

The TF-IDF transformation does not overcome the fundamental issue that semantically similar words, such as "man" and "woman", and semantically dissimilar words, such as "man" and "banana", have the same distance (Hamming distance of 1).

Word Embeddings

For one-hot vector $x_{nt} \in \{0,1\}^V$[7], we define

$$e_{nt} := Ex_{nt} \in \mathbb{R}^K \qquad \text{(word embeddings)}$$

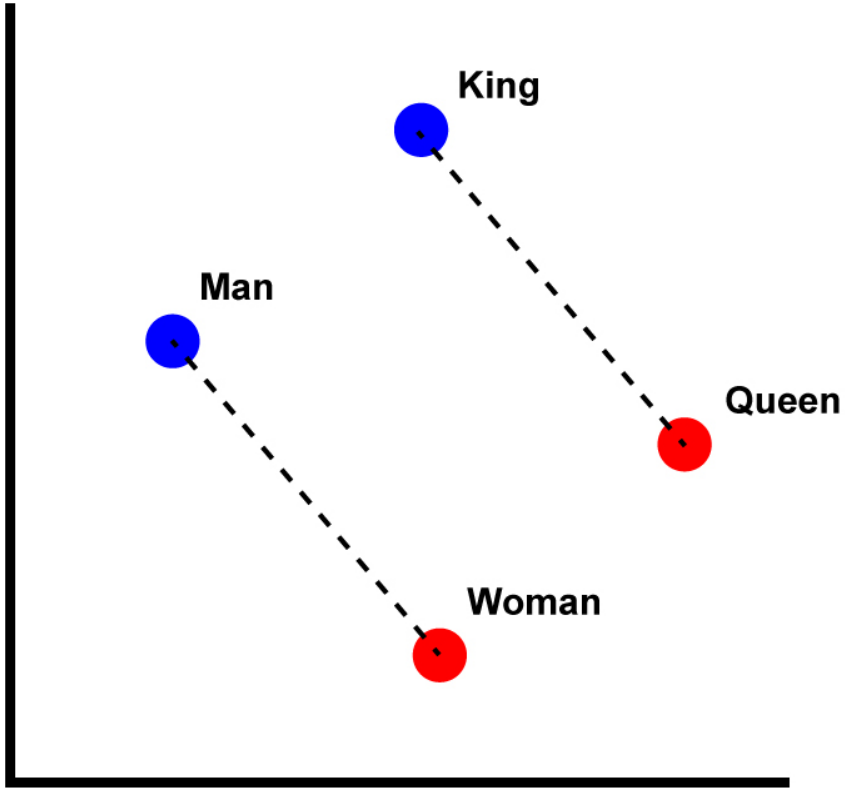
$$\bar{e}_n := \sum_{t=1}^{|x_n|}{}^{8} e_{nt} = E \cdot BOW(x_n) \qquad \text{(bag of word embeddings)}$$

where $E = [e(v_1)\ e(v_2)\ \cdots\ e(v_V)] \in \mathbb{R}^{K \times V}$ and $e(v_i) \in \mathbb{R}^K$

- $K < V$
- e.g., $p(y = c \mid x_n; \theta) = \text{softmax}_c(W\bar{e}_n)$ (multi-class logistic regression)

[7] a standard basis vector

[8] $\sim$ global average pooling



algebraic operations on the vector representations of words (word2vec [MCC+13])

Word Embeddings

semantically similar = occur in similar contexts (distributional hypothesis) = close in vector space

- Word2vec (CBOW, skipgram)
- ...
- contextual word embeddings (ELMo, BERT, GPT, T5, ...): The contextual embedding of a word is a function of all the words in its context (usually a sentence). This can give much improved results, and is currently the standard approach to representing natural language data, as a preprocessing step before doing transfer learning.

Short Introduction to Probabilistic Graphical Models

Probabilistic Graphical Model (PGM)

A PGM is a joint distribution using a graph (DAG) to encode conditionally independent (CI) assumption:

$$Y_i \perp Y_{pred(i) \setminus pa(i)} \mid Y_{pa(i)},$$

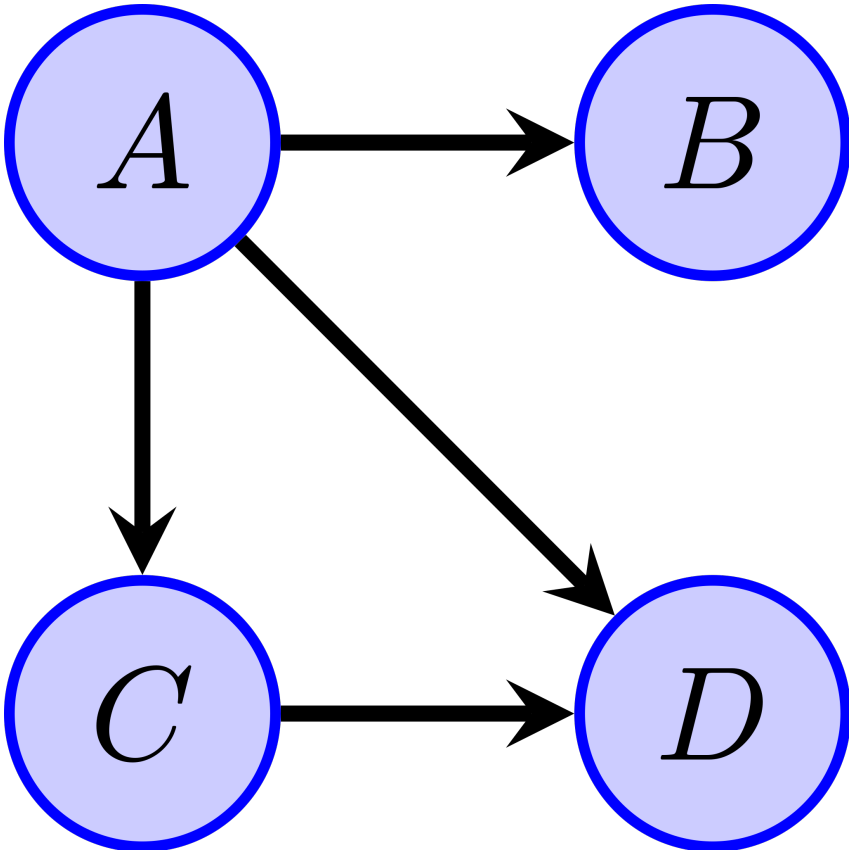
where $pred(i)$ and $pa(i)$ are the predecessors and parents of node i, respectively.

- graph: node (random variable), directed edge (dependency), lack of edge (CI)

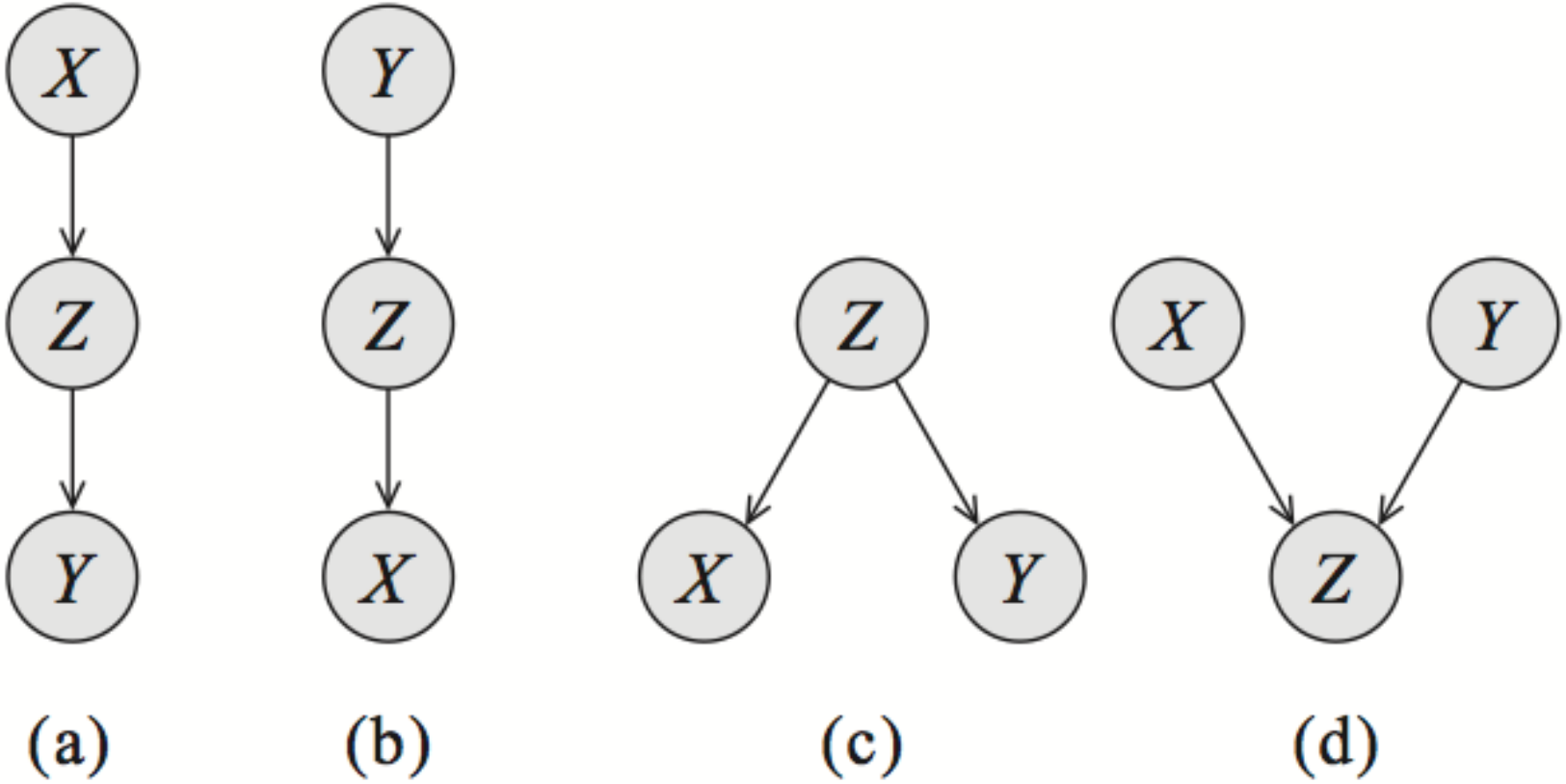
Consequently, we can represent the joint distribution as follows:

$$p(Y_{1:V}) = \prod_{i=1}^{V} p(Y_i \mid Y_{pa(i)})$$

Q. Compute the $p(A, B, C, D)$ for the following graph:

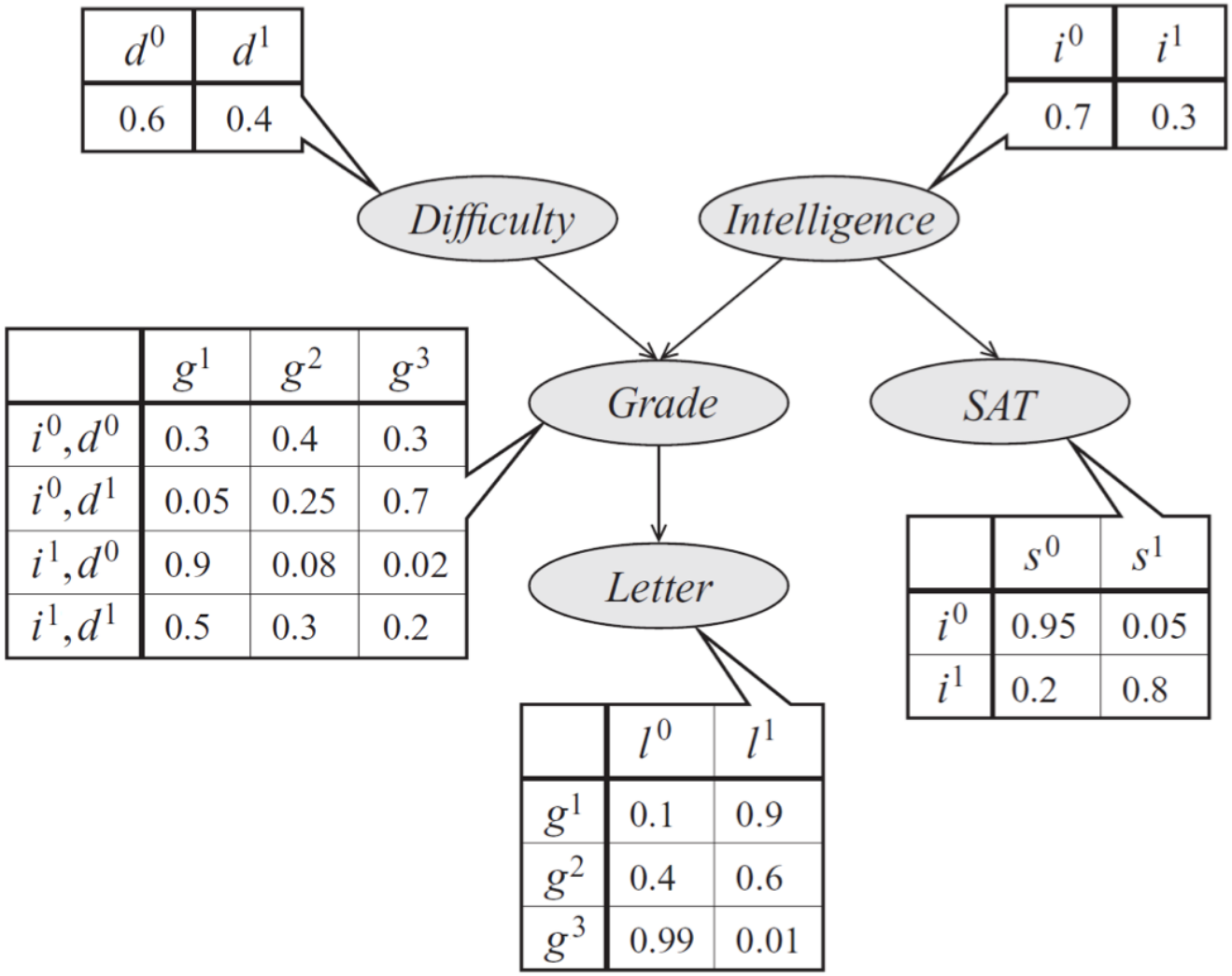


Q. Compute the $p(X, Y, Z)$ for the following graph:



cf. (a,b) $X \perp Y \mid Z$

Q. Compute the $p(D, I, G, S, L)$ for the following graph:



d^0	d^1
0.6	0.4

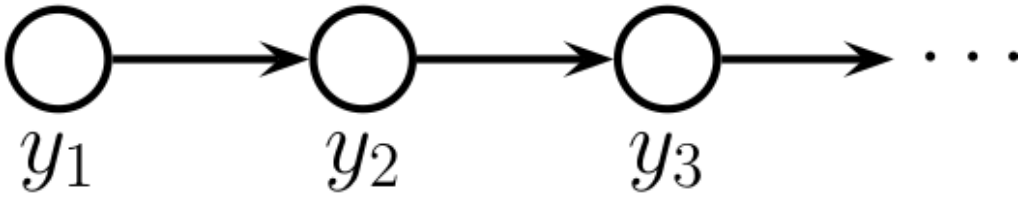
i^0	i^1
0.7	0.3

	g^1	g^2	g^3
i^0, d^0	0.3	0.4	0.3
i^0, d^1	0.05	0.25	0.7
i^1, d^0	0.9	0.08	0.02
i^1, d^1	0.5	0.3	0.2

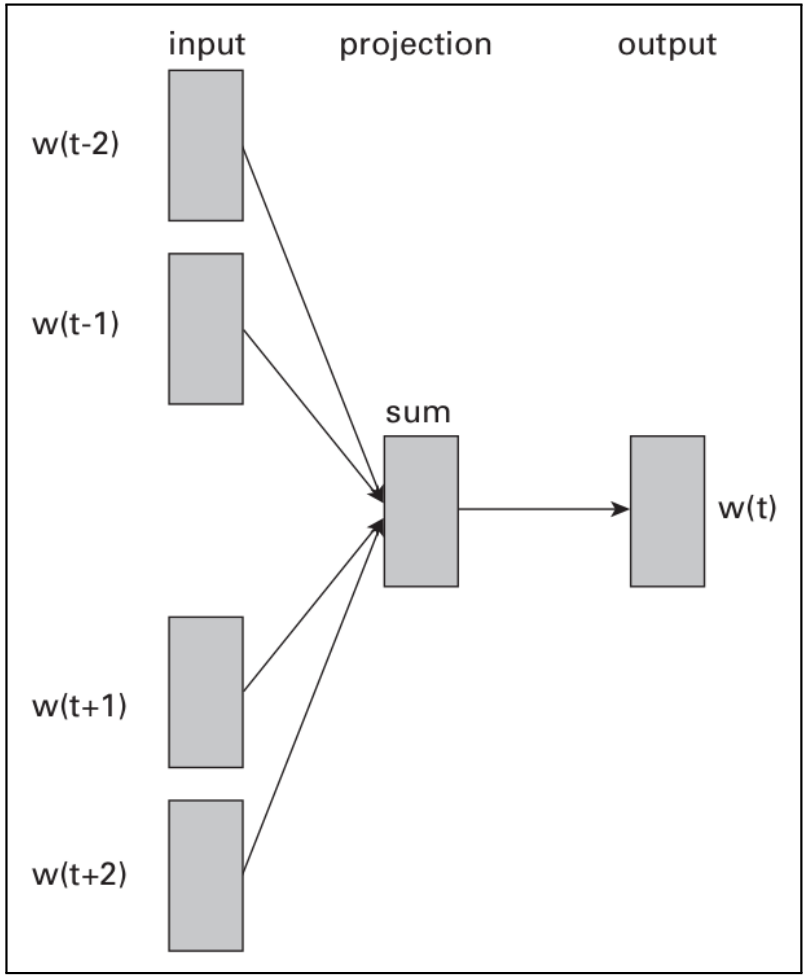
	s^0	s^1
i^0	0.95	0.05
i^1	0.2	0.8

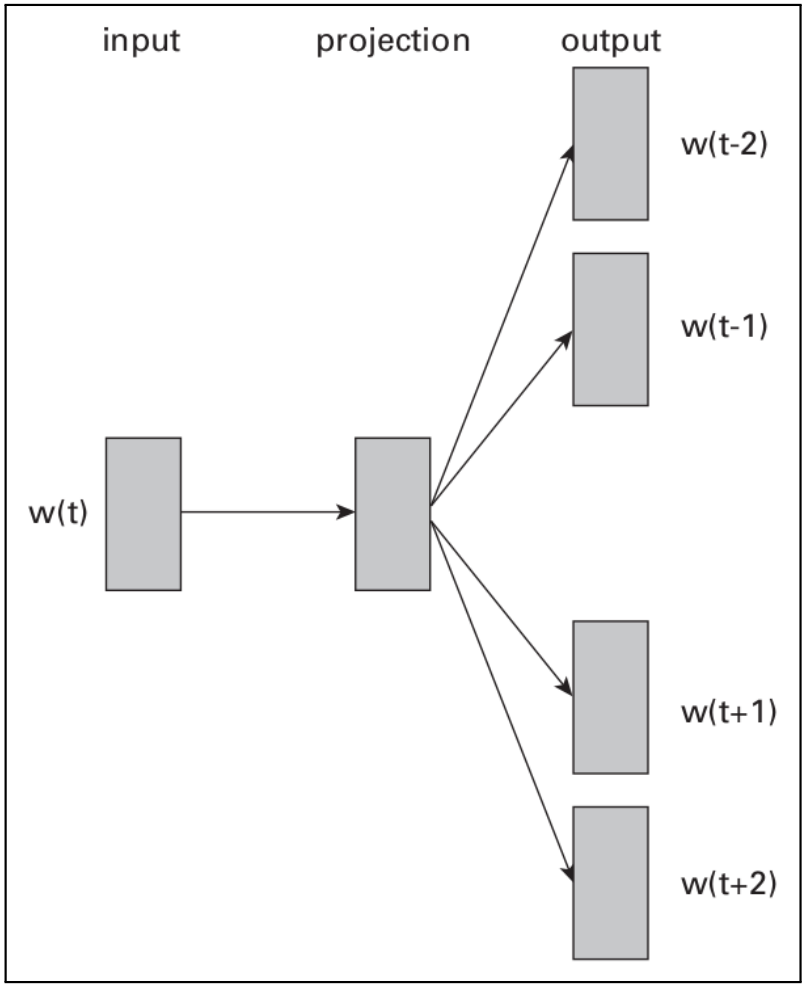
	l^0	l^1
g^1	0.1	0.9
g^2	0.4	0.6
g^3	0.99	0.01

Q. Compute the $p(y_1, y_2, \cdots, y_T)$ for the following graph:



Q. Compute the $p(w_{t-2}, w_{t-1}, w_t, w_{t+1}, w_{t+2})$ for the following graphs:





Word2vec [MCC+13]

shallow NNs (u or v) for predicting a word given its context

- CBOW (continuous BOW); $\bar{v}_t$ ($\sim$ BOW) as a context

$$\log p(w_{1:T}) = \sum_{t=1}^{T} \log p(w_t \mid w_{J_t}) \qquad (J_t = [t-m, t+m] \backslash \{t\})$$

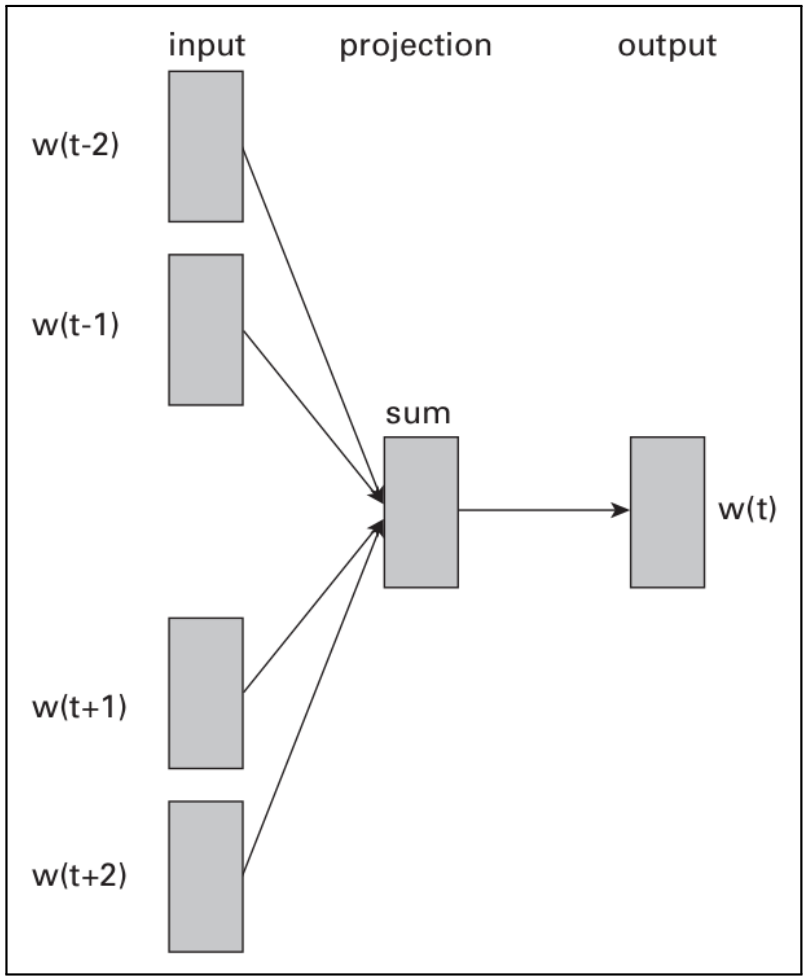
$$= \sum_{t=1}^{T} \log \frac{\exp(v(w_t)^\top \bar{v}_t)}{\sum_{w'} \exp(v(w')^\top \bar{v}_t)}$$

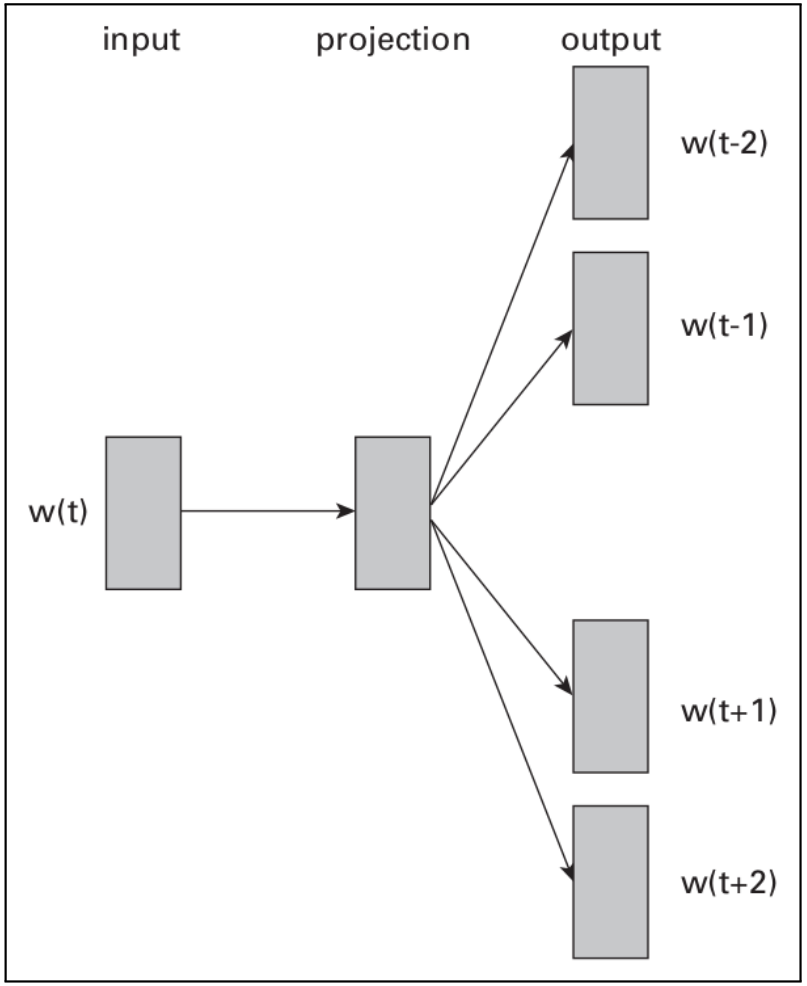
$$\bar{v}_t = \frac{1}{2m} \sum_{j \in J_t} v(w_j)$$

- skipgram (u: input embedding, v: target embedding)

$$\log p(w_{1:T}) = \sum_{t=1}^{T} \sum_{j \in J_t} \log p(w_j \mid w_t)$$

$$= \sum_{t=1}^{T} \sum_{j \in J_t} \log \frac{\exp(v(w_j)^\top u(w_t))}{\sum_{w'} \exp(v(w')^\top u(w_t))}$$





$$\log \frac{\exp(v(w_t)^\top \bar{v}_t)}{\sum_{w'} \exp(v(w')^\top \bar{v}_t)} \text{ and } \sum_{j \in J_t} \log \frac{\exp(v(w_j)^\top u(w_t))}{\sum_{w'} \exp(v(w')^\top u(w_t))}$$

$$\text{context } (\bar{v}_t \text{ and } u(w_t)) \mapsto \text{prediction } (v(w_t) \text{ and } \{v_j\}_{j \in J_t})$$

- e.g., $V = 10000, K = 16, N = 25k$ (IMDB)
- $E \in \mathbb{R}^{16 \times 10000}$
- instead of learning the whole parameters in a supervised way, we can perform unsupervised pre-training of word embedding models (E)
- then, fine-tune the other parameters for a downstream task

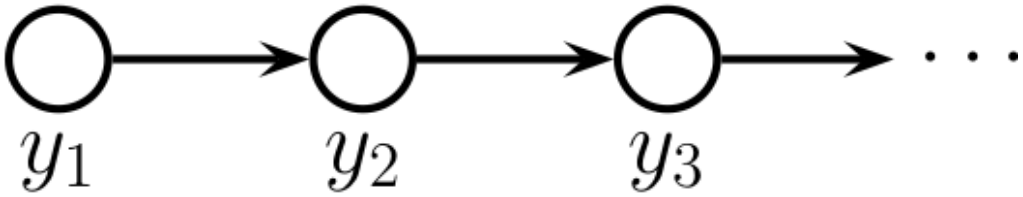
Lecture 16

Recurrent NNs (RNNs)

- Vec2Seq ($x \mapsto y_{1:T}$)
- Seq2Vec ($x_{1:T} \mapsto y$)
- Seq2Seq ($x_{1:T} \mapsto y_{1:T'}$)

These models are trained in a supervised way ($L(\theta) = \text{NLL}$) with given input-output pairs.

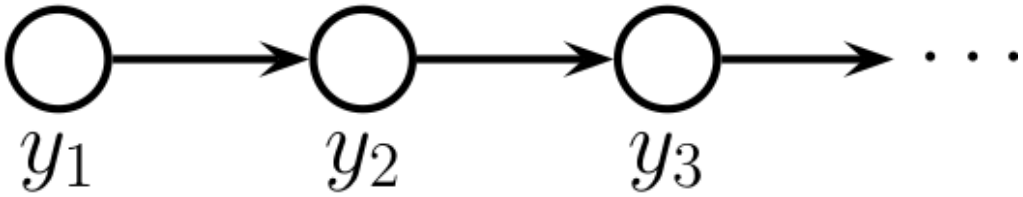
Markov Model (Markov Chain)



(Wiki)
"A Markov model is a stochastic model used to model pseudo-randomly changing systems. It is assumed that **future states depends only on the current state, not on the events that occurred before it** (that is, it assumes **the Markov property**)."

- e.g., hidden Markov model (HMM) - ... many applications to pattern recognition
- The Markov assumption is too strong (inductive bias).

Markov Model (Markov Chain)



We want to create a joint probability distribution over variable-length sequences, $p(y_{1:T})$ (e.g, sentences). Generally, we have

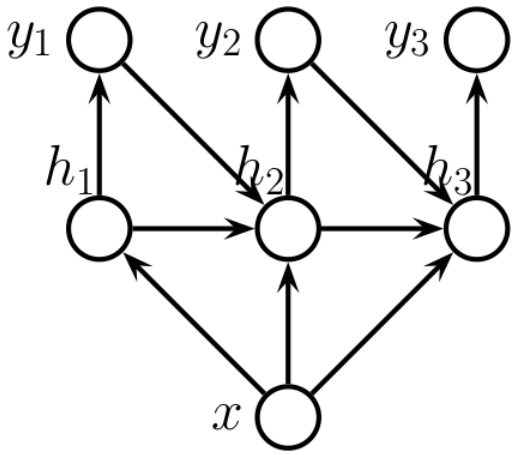
$$p(y_{1:T}) = p(y_1)p(y_2 \mid y_1)p(y_3 \mid y_2, y_1)\cdots = \prod_{t=1}^{T} p(y_t \mid y_{1:t-1})$$

and, under the Markov assumption, we have

$$p(y_{1:T}) = p(y_1)p(y_2 \mid y_1)p(y_3 \mid y_2)p(y_4 \mid y_3)\cdots = \prod_{t=1}^{T} \underbrace{p(y_t \mid y_{t-1})}_{\text{Markov kernel}}$$

where $y_0 = \phi$.

Vec2Seq



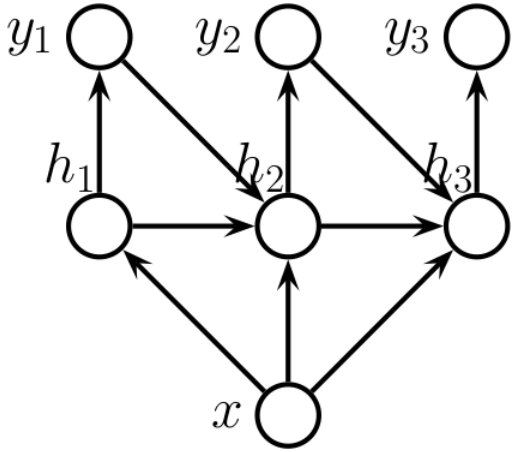
$$p(h_1 \mid x) \qquad \text{(often deterministic)}$$

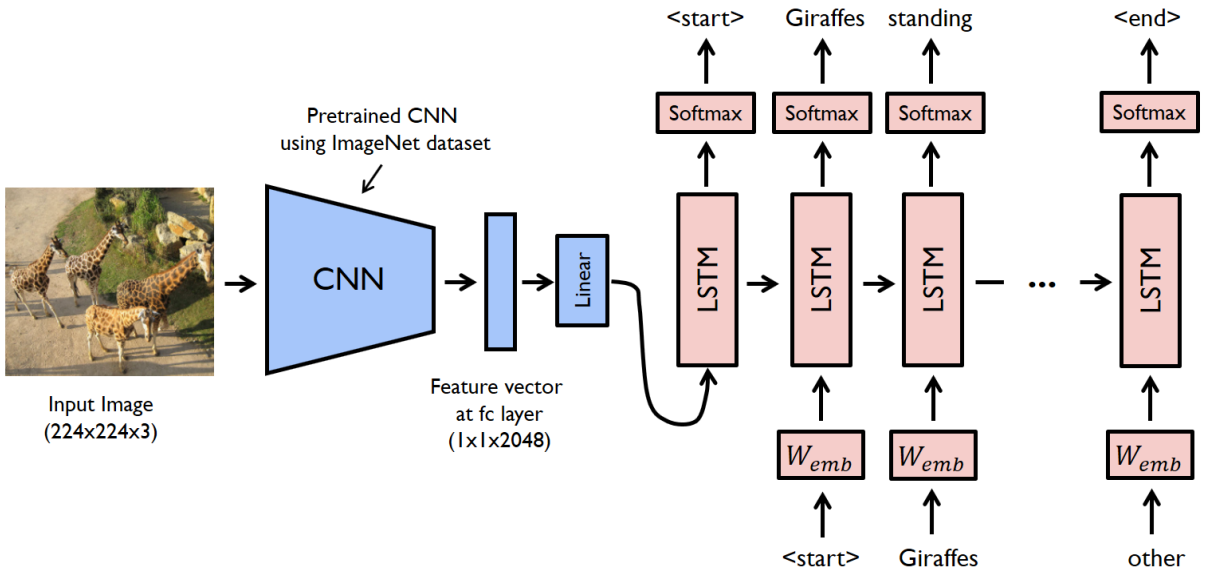
$$p(y_t \mid h_t) = Cat(y_t \mid \text{softmax}(W' h_t)) \qquad \text{(stochastic)}$$

$$\underset{\substack{\downarrow \\ \text{hidden state} \\ \text{for } y_{t+1}}}{h_{t+1}} = \phi(W[x; y_t; h_t])^{9}$$

- y_t depends on h_t ("in a stateful way"), which depends on y_{t-1}, which depends on h_{t-1}, and so on, Thus y_t implicitly depends on all past observations. Thus an RNN overcomes the limitations of standard Markov models, in that they can have unbounded memory.

[9] The textbook notation is equivalent to ours ($W = [W_{xh}\ W_{hh}]$).

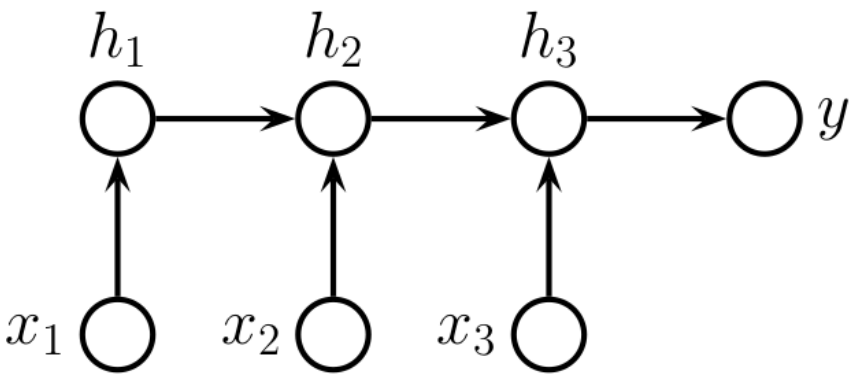




applications:

- language modeling (unconditional seq. generation); $x = \phi$
- image captioning; $x =$ some embedding of the image captured by a ConvNet

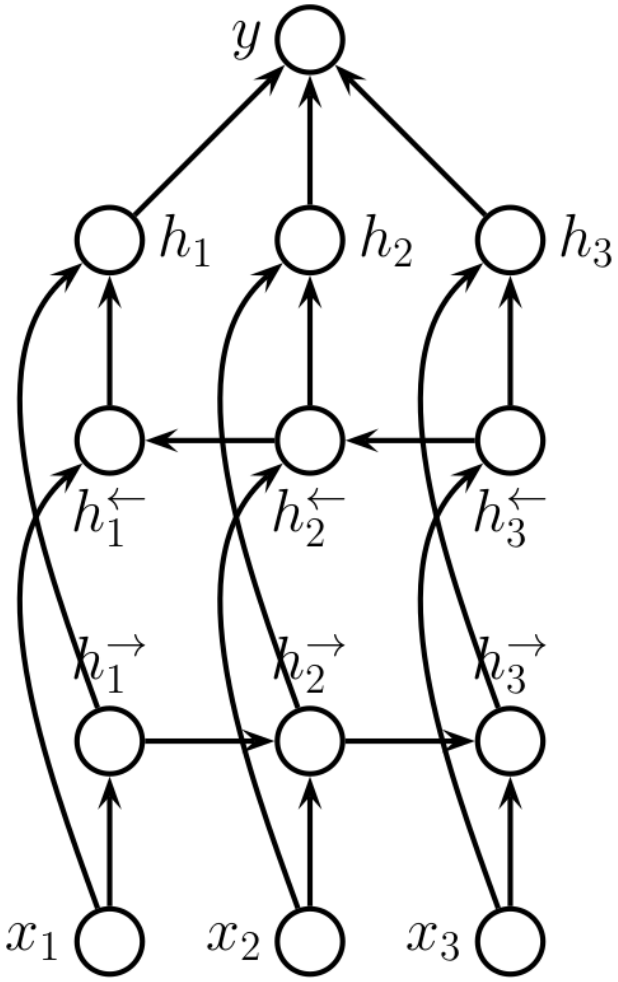
Seq2Vec



$$h_t = \phi(W[x_t; h_{t-1}])$$
$$p(y \mid x_{1:T}) = Cat(y \mid \text{softmax}(W' h_T))$$

- e.g., sequence classification

Seq2Vec - Bidirectional RNN

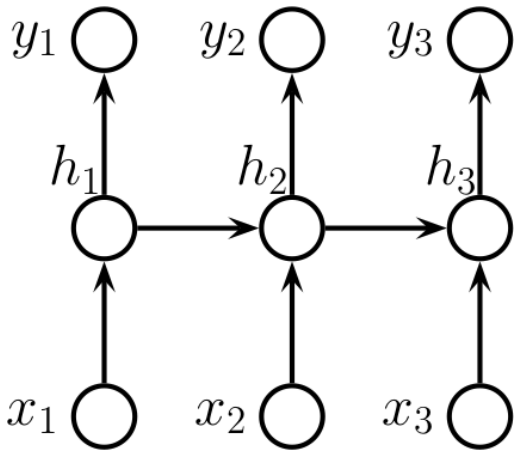


We can often get better results if we let the hidden states depends on the past and future context (bidirectional RNN) as follows:

$$h_t^{\rightarrow} = \phi(W^{\rightarrow}[x_t; h_{t-1}^{\rightarrow}])$$
$$h_t^{\leftarrow} = \phi(W^{\leftarrow}[x_t; h_{t-1}^{\leftarrow}])$$
$$h_t = [h_t^{\rightarrow}; h_t^{\leftarrow}]$$
$$\bar{h} = \frac{1}{T}\sum_{t=1}^{T} h_t \text{ (avg. pooling)}$$
$$p(y \mid x_{1:T}) = Cat(y \mid \text{softmax}(W'\bar{h}))$$

Seq2Seq

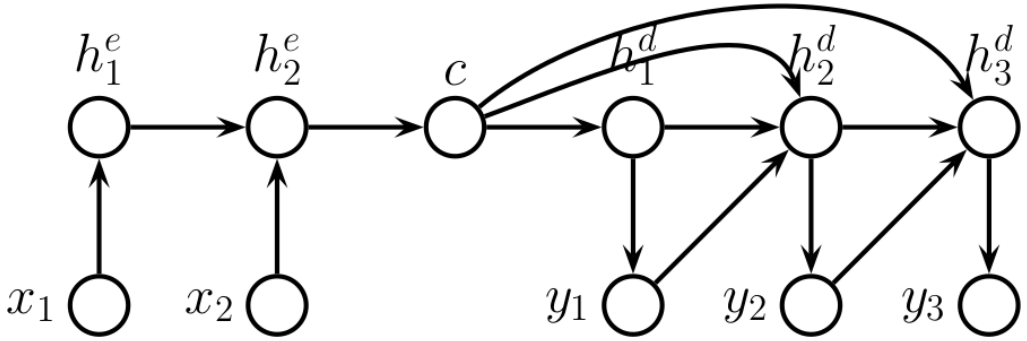
aligned case ($T = T'$, e.g., dense seq. labeling)



$$h_t = \phi(W[x_t; h_{t-1}])$$
$$p(y_t \mid h_t) = Cat(y_t \mid \text{softmax}(W' h_t))$$

Seq2Seq [SVL14; CMG+14]

unaligned case ($T \neq T'$) - encoder-decoder architecture



$$h_t^e = \phi(W^e[x_t; h_{t-1}^e])$$

$$c = h_T^e \qquad \text{(Seq2Vec)}$$

$$h_t^d = \phi(W^d[c; y_{t-1}; h_{t-1}^d]) \qquad \text{(Vec2Seq)}$$

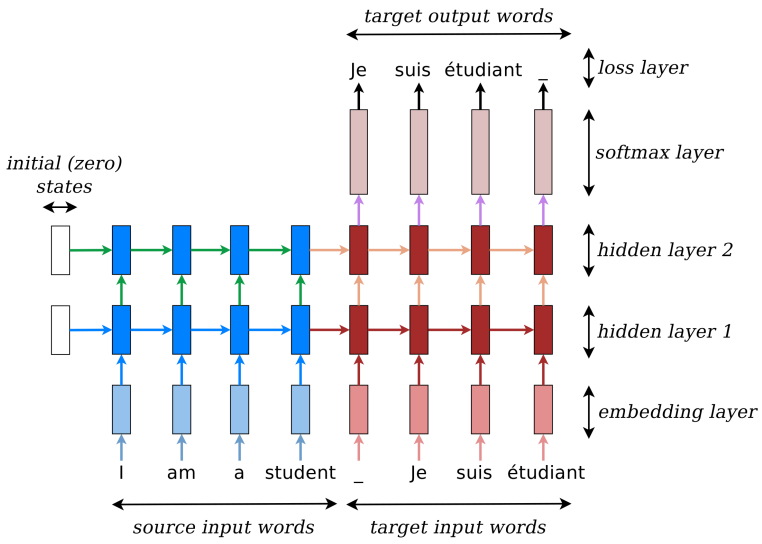
$$p(y_t \mid h_t^d) = Cat(y_t \mid \text{softmax}(W' h_t^d))$$

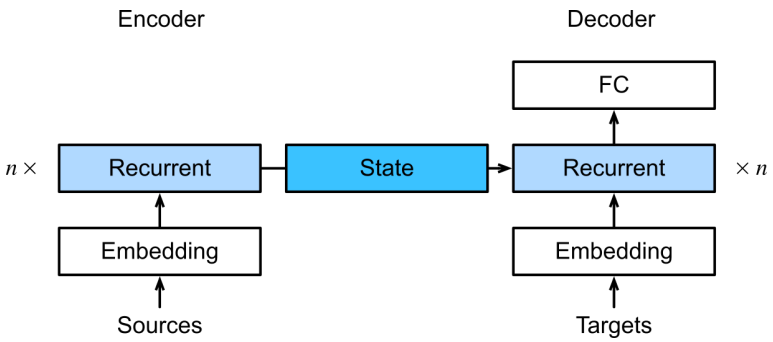
applications:

- neural machine translation (NMT)
- document summarization
- question answering

Going Deeper ($\times n$)

We can create more expressive models by stacking multiple hidden chains on top of each other ($h_t^0 := x_t$).





$$h_t^\ell = \phi(W^\ell[\underbrace{h_t^{\ell-1}}_{x_t}; h_{t-1}^\ell])$$

Teacher Forcing

likelihood of a sequence of words $w_1, w_2, \cdots, w_T$ (ground truth):

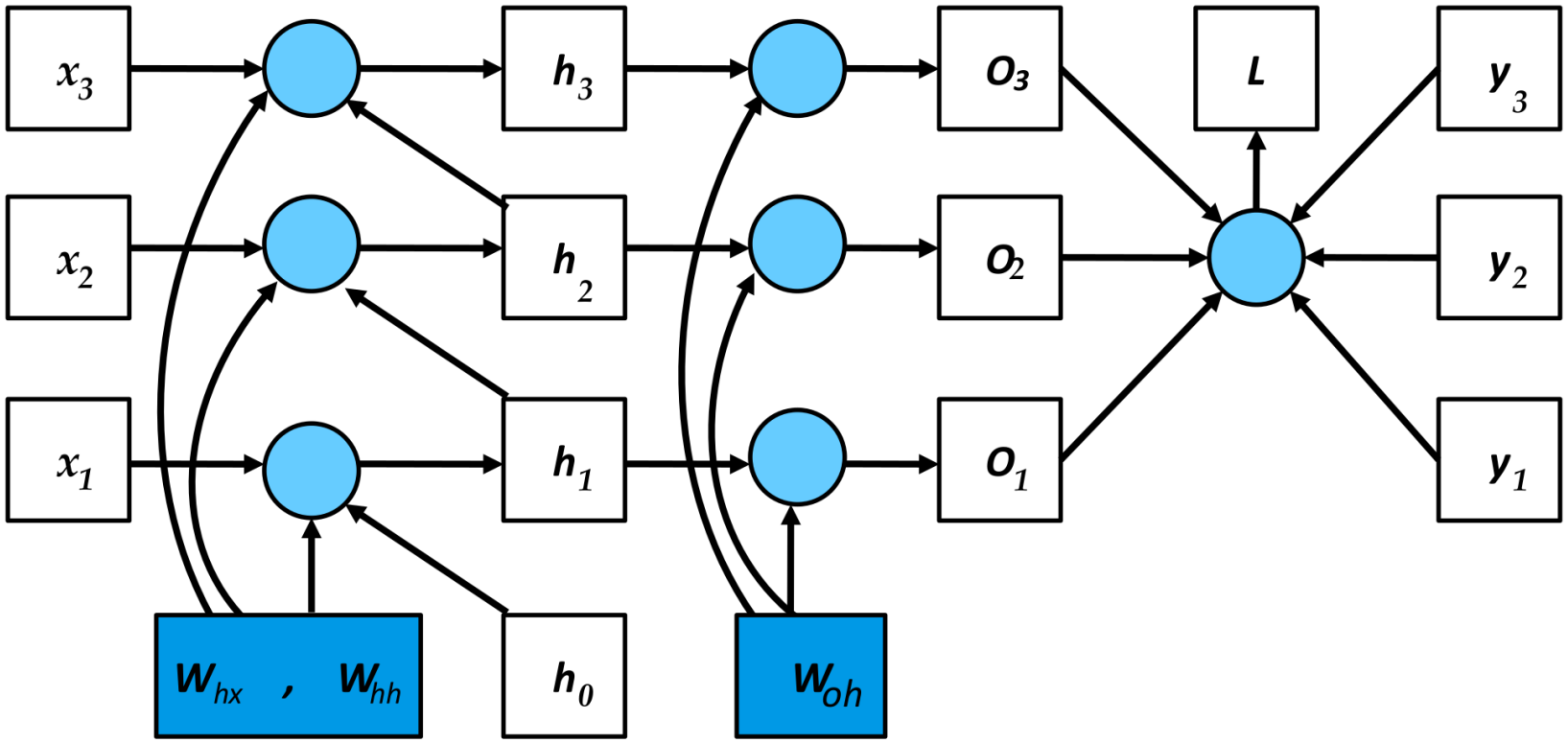
$$p(w_{1:T}) = \prod_{t=1}^{T} p(w_t \mid w_{1:t-1}).$$

We condition on the ground truth labels from the past, $w_{1:t-1}$, not labels generated from the model. This is called teacher forcing, since the teacher's values are "force fed" into the model as input at each step.

- right-shifted input
- e.g., $p(w_1) = p(w_1 \mid \text{None})$

Backpropagation Through Time (BPTT)

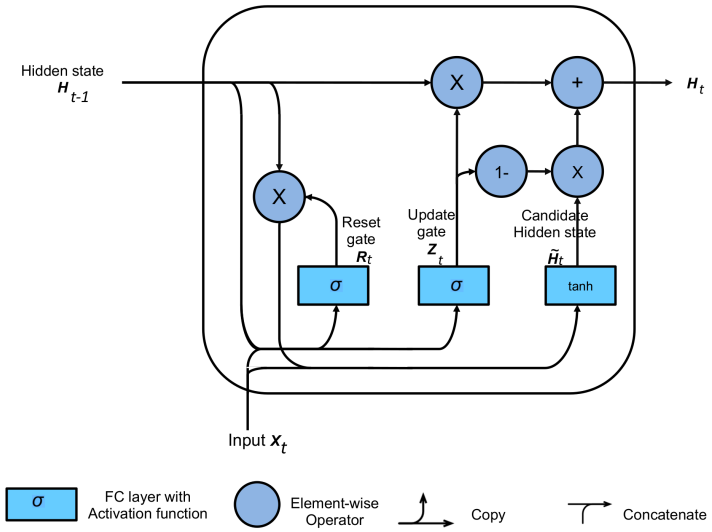
We can unroll the computation graph and then apply the backpropagation algorithm (BPTT) to compute gradients of the loss wrt the parameters.

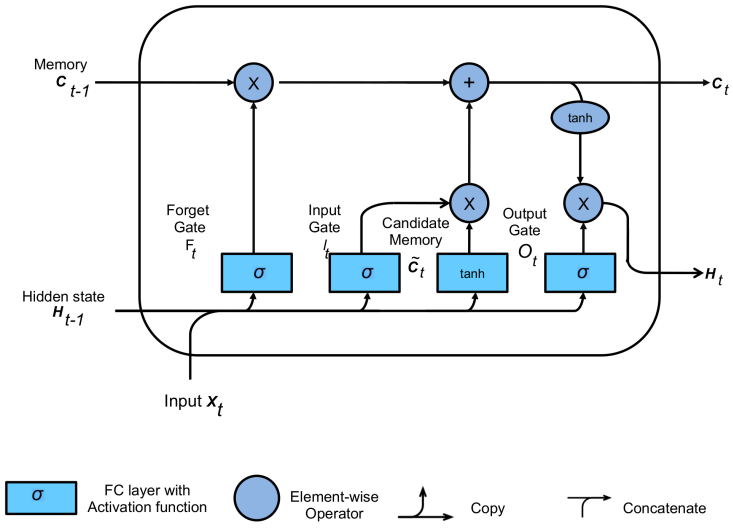


GRU and LSTM

In practice, "vanilla" RNNs fail to remember inputs from long in the past, because of the vanishing gradient problem. As a solution to this, we update the hidden state in an additive way, similar to residual net.

- Gated Recurrent Units (GRU) [CMG+14]
- Long Short Term Memory (LSTM) [HS97]





Lecture 17

Understanding Language

- NNs for Language
- Transformer
- LLM

Attention[11]

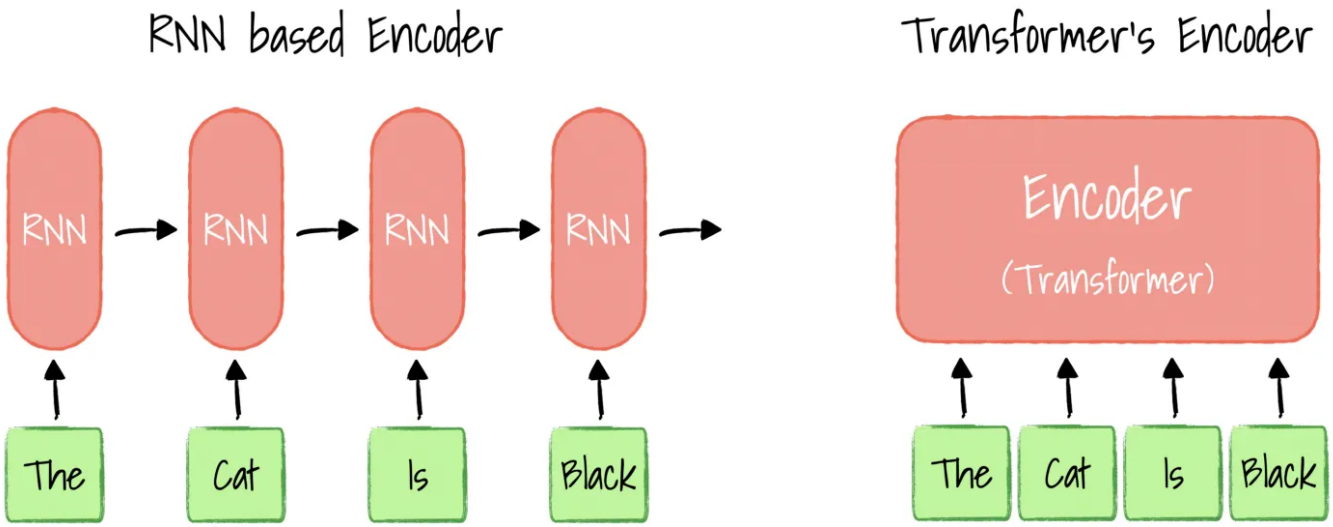
In RNNs, hidden activations (recurrent layers) are usually written as

$$h_t^\ell = \phi(W^\ell[h_t^{\ell-1}; h_{t-1}^\ell]).$$

However, we can imagine a more flexible (less inductive bias[10]) hypothesis space for an intermediate representation mapping:

$$h^\ell = A(h^{\ell-1})h^{\ell-1}$$

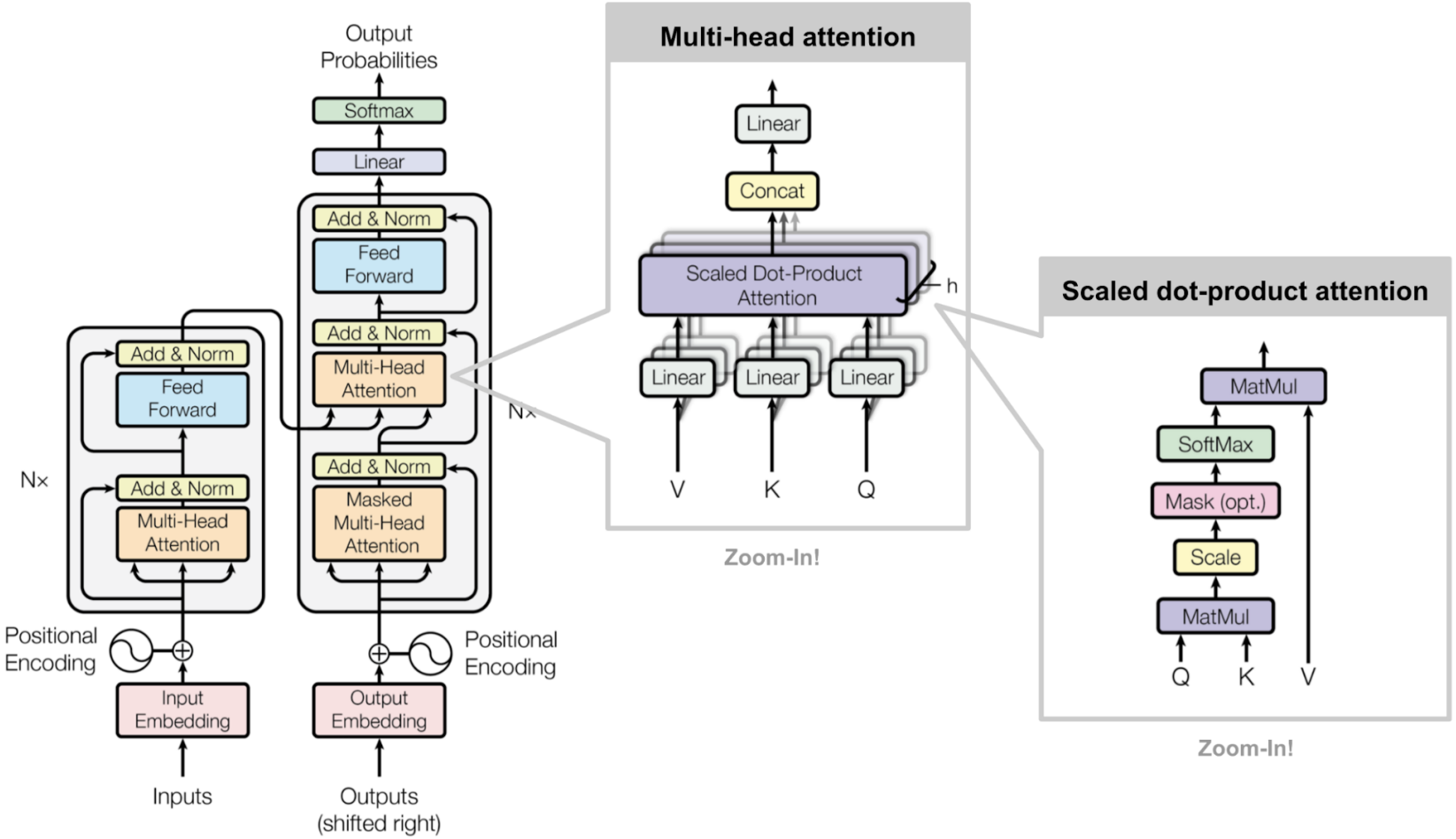
$$\underbrace{h_i^\ell}_{\text{row}} = \underbrace{\alpha_{i*}(h^{\ell-1})}_{\text{sum to 1}} h^{\ell-1}$$



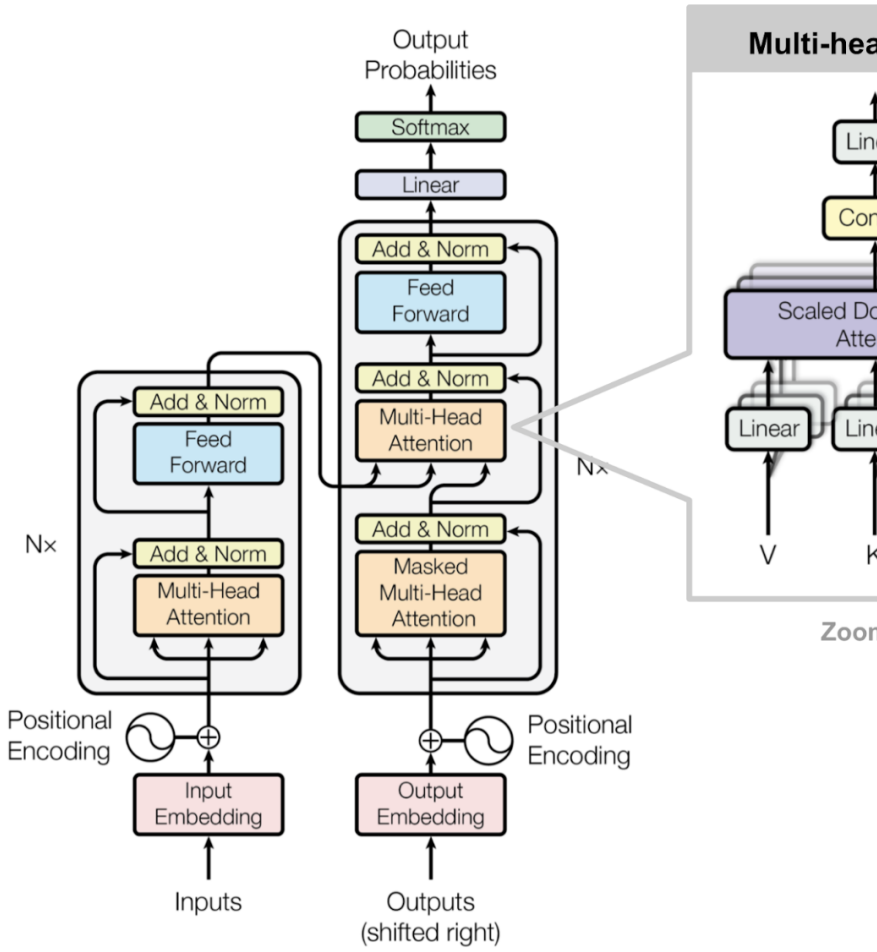
[10] Markov models > RNNs > attention

[11] ... is all you need [VSP+17]

Transformer [VSP+17]



- Input/Output Embedding ✓
- Positional Encoding
- Multi-Head Attention
- Add & Norm
- Feed Forward
- Decoder
  - shifted right
  - Masked Multi-Head Attention
  - Cross Attention



Records, Fields, and Keys

Table: Relational Database, Data Table. We highlight a record with a red row.

Key Field	Field1	Field2	Field3	Field4
101	$\cdots$	$\cdots$	$\cdots$	$\cdots$
102	$\cdots$	$\cdots$	$\cdots$	$\cdots$
104	$\cdots$	$\cdots$	$\cdots$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
1200	$\cdots$	$\cdots$	$\cdots$	$\cdots$

a key $\mapsto$ location of the corresponding record

Attention as Soft Dictionary Lookup

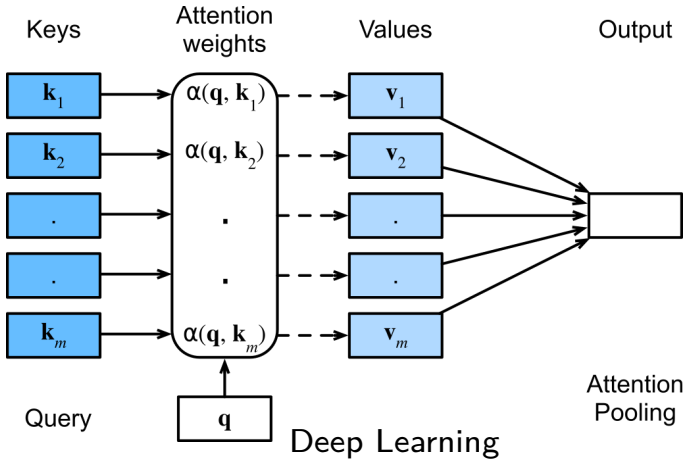
We compare the query q to each key k_j and then retrieve the corresponding value v_j (in a hard way ($\in \{0, 1\}$))

$$\sum_{j=1}^{N} \underbrace{\mathbf{1}(q = k_j)}_{\text{hard}} v_j.$$

To make this lookup operation differentiable, we compute a soft ($\in [0, 1]$) combination of v_j's:

$$\sum_{j=1}^{N} \underbrace{[\alpha(q, k_{1:N})]_j}_{\text{soft}} v_j$$

where $[\alpha(q, k_{1:N})]_j$ computes the relative similarity of query q to key k_j among a set of keys $k_{1:N}$.



Attention Weight

$$
\begin{aligned}
A &= (\alpha_{ij}) \\
\alpha_{ij} &\equiv [\alpha(q_i, k_{1:N})]_j \\
&:= \underbrace{\mathrm{softmax}_j(a(q_i, k_1), a(q_i, k_2), \cdots, a(q_i, k_N))}_{\substack{\text{the relative similarity of query } q_i \text{ to key } k_j \\ \text{among a set of keys } k_{1:N}}}
\end{aligned}
$$

where $a(q, k)$ measure the similarity of query q to key k, and the sum of α_{i*} (row-wise sum) satisfies $\sum_j \alpha_{ij} = 1$.

Similarity: scaled dot-product attention

We use

$$a(q,k) = q^\top k/\sqrt{d_k} \qquad \text{(scaled dot-product attention)}$$ [12]

where $q, k \in \mathbb{R}^{d_k}$. Then, we can simply write

$$\begin{aligned}
A &= \text{softmax}(QK^\top/\sqrt{d_k}) && \text{((row-wise) softmax)} \\
\alpha_{i*} &= \text{softmax}([q_i^\top k_1, q_i^\top k_2, \cdots, q_i^\top k_N]/\sqrt{d_k}) \\
[QK^\top]_{ij} &= e_i^\top [QK^\top] e_j \\
&= [Q^\top e_i]^\top [K^\top e_j] \\
&= q_i^\top k_j
\end{aligned}$$

[12] $a(q,k) = w_v^\top \tanh(W_q q + W_k k)$: additive attention

Kernel Regression[15] as Non-parametric Attention

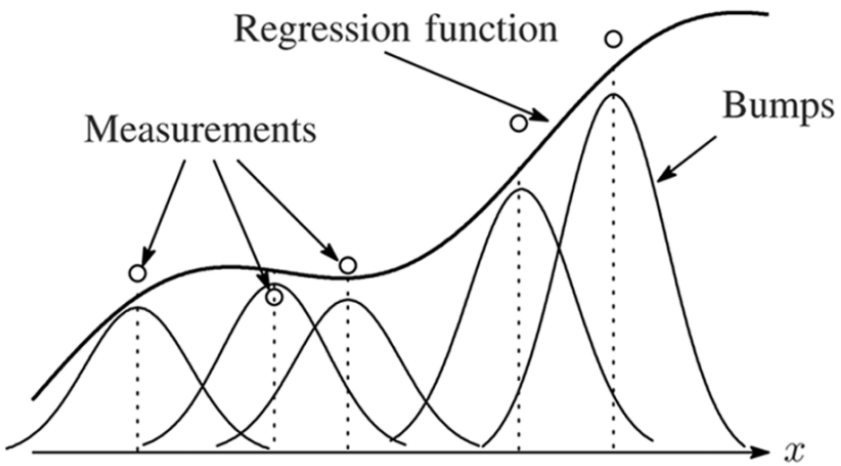


Figure: [TFM07]

The predicted output $f(x)$ (regression function) at query point x is $f(x) = \sum_j \alpha_j y_j$ (sum of bumps), a weighted combination of all the target labels y_j (measurements), where the weights α_j depend on the similarity of query point x to each training point x_j.[13][14]

[13] $\alpha_j \equiv [\alpha(x, x_{1:N})]_j, x = q, x_j = k_j, y_j = v_j$

[14] For example, $[\alpha(x, x_{1:N})]_j \propto \exp(-\|x - x_j\|^2)$

[15] Nadaraya-Watson kernel regression (α_j's sum to 1)

$$h = \left[h_1^\top; h_2^\top; \cdots; h_N^\top\right] \in \mathbb{R}^{N\times d}$$

$$V(h) = h \underbrace{W^V}_{d\times d_v} = \left[v_1^\top; v_2^\top; \cdots; v_N^\top\right] \in \mathbb{R}^{N\times d_v}$$

$$\text{head} = \underbrace{A(h)}_{N\times N} \underbrace{V(h)}_{N\times d_v} \in \mathbb{R}^{N\times d_v}$$

$$= \begin{bmatrix} \alpha_{11} & \alpha_{12} & \cdots & \alpha_{1N} \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} v_1^\top \\ v_2^\top \\ \vdots \\ v_N^\top \end{bmatrix} \in \mathbb{R}^{N\times d_v}$$

$$= \begin{bmatrix} \sum_j \alpha_{1j}{}^{16} v_j^\top \\ \vdots \end{bmatrix} \in \mathbb{R}^{N\times d_v}$$

[16] softly and dynamically decides (in an input dependent way) which one ($v_j^\top$) to use

Attention (General)

$$
\begin{aligned}
\text{head} &= \text{Attention}(QW^Q, KW^K, VW^V) \\
&= \underbrace{\phi_s(QW^Q W^{K\top} K^\top / \sqrt{d_k})}_{N \times N} \underbrace{VW^V}_{N \times d_v}
\end{aligned}
$$

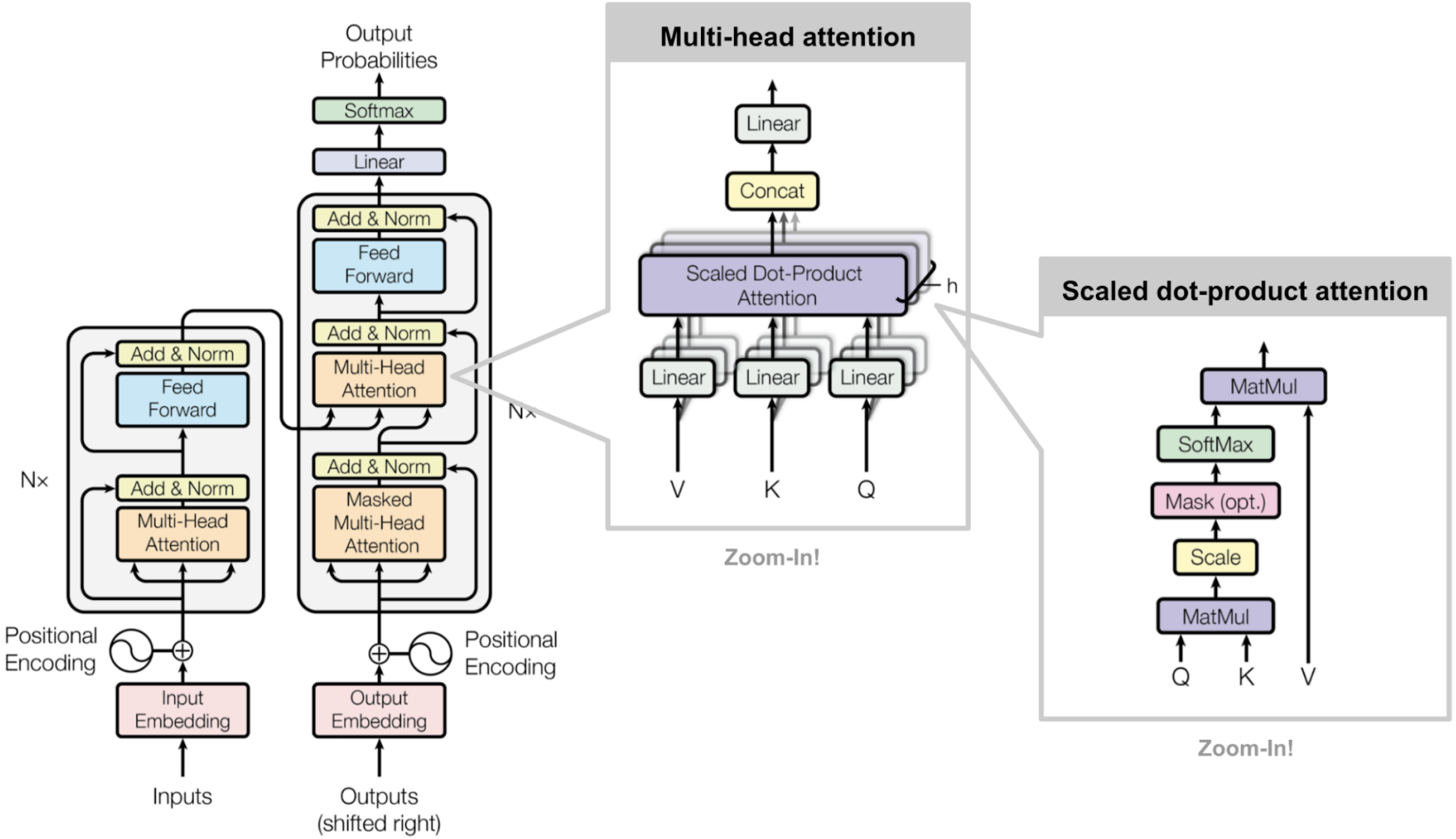
Multi-Head Attention (General)

$$
\begin{aligned}
\mathrm{MHA}(Q, K, V) &= \mathrm{Concat}(\mathrm{head}_1 \ \mathrm{head}_2 \ \cdots \ \mathrm{head}_h) \ \underbrace{W}_{hd_v \times d_o} \\
&= [\underbrace{\mathrm{head}_1}_{N \times d_v} \ \mathrm{head}_2 \ \cdots \ \mathrm{head}_h][W_1; \ W_2; \ \cdots; \ W_h]^{17} \\
&= \sum_i \underbrace{\mathrm{head}_i}_{N \times d_v} \ \underbrace{W_i}_{d_v \times d_o} \\
&= \sum_i \underbrace{\mathrm{Attention}(QW_i^Q, KW_i^K, VW_i^V)}_{N \times d_v} \ \underbrace{W_i}_{d_v \times d_o} \\
&= \sum_i \underbrace{\phi_s(Q\Sigma_i K^\top)}_{N \times N} VW_i^V W_i
\end{aligned}
$$

$\Sigma_i (= W_i^Q W_i^{K\top} \in \mathbb{R}^{d \times d})$'s are used to capture different notions of similarity.

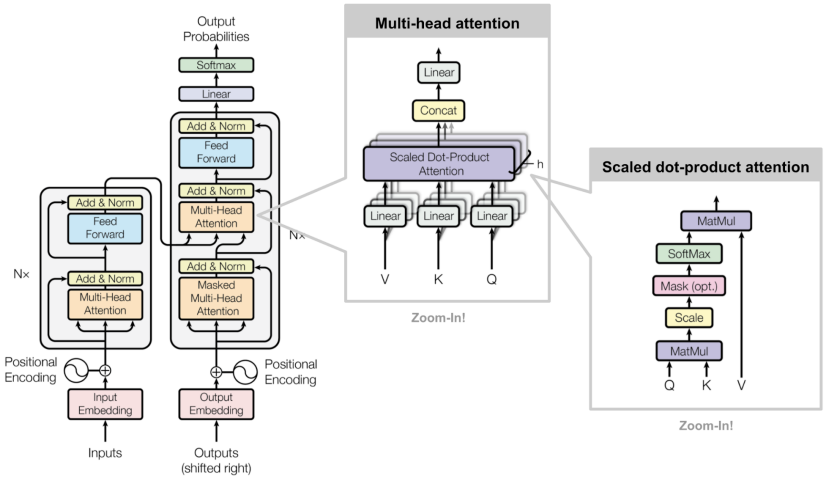
[17] The number of heads h is often 8.

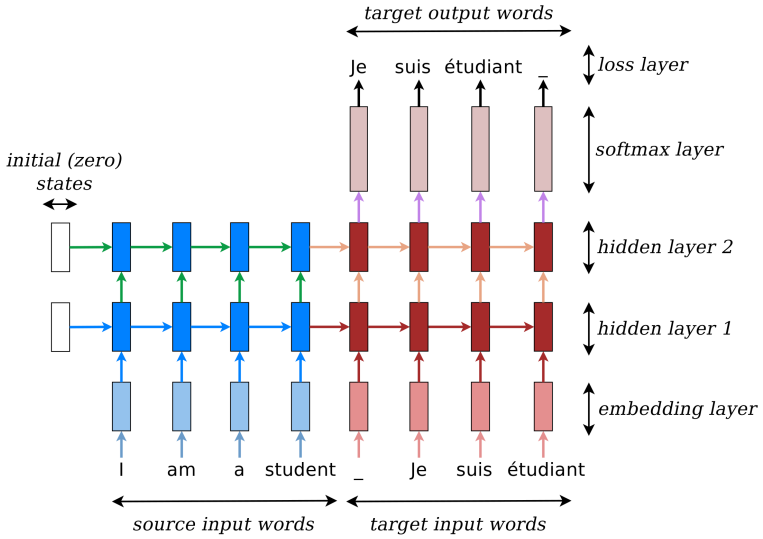
Transformer [VSP+17]

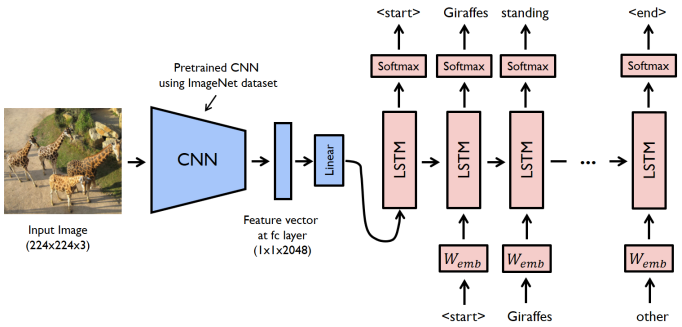


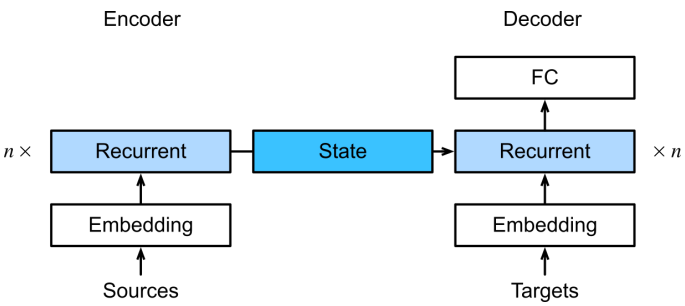
- Input/Output Embedding ✓
- Positional Encoding
- Multi-Head Attention ✓; $\mathrm{MHA}(X, X, X)$
- Add & Norm
- Feed Forward
- Decoder
  - shifted right
  - Masked Multi-Head Attention
  - Cross Attention

Transformer [VSP+17]





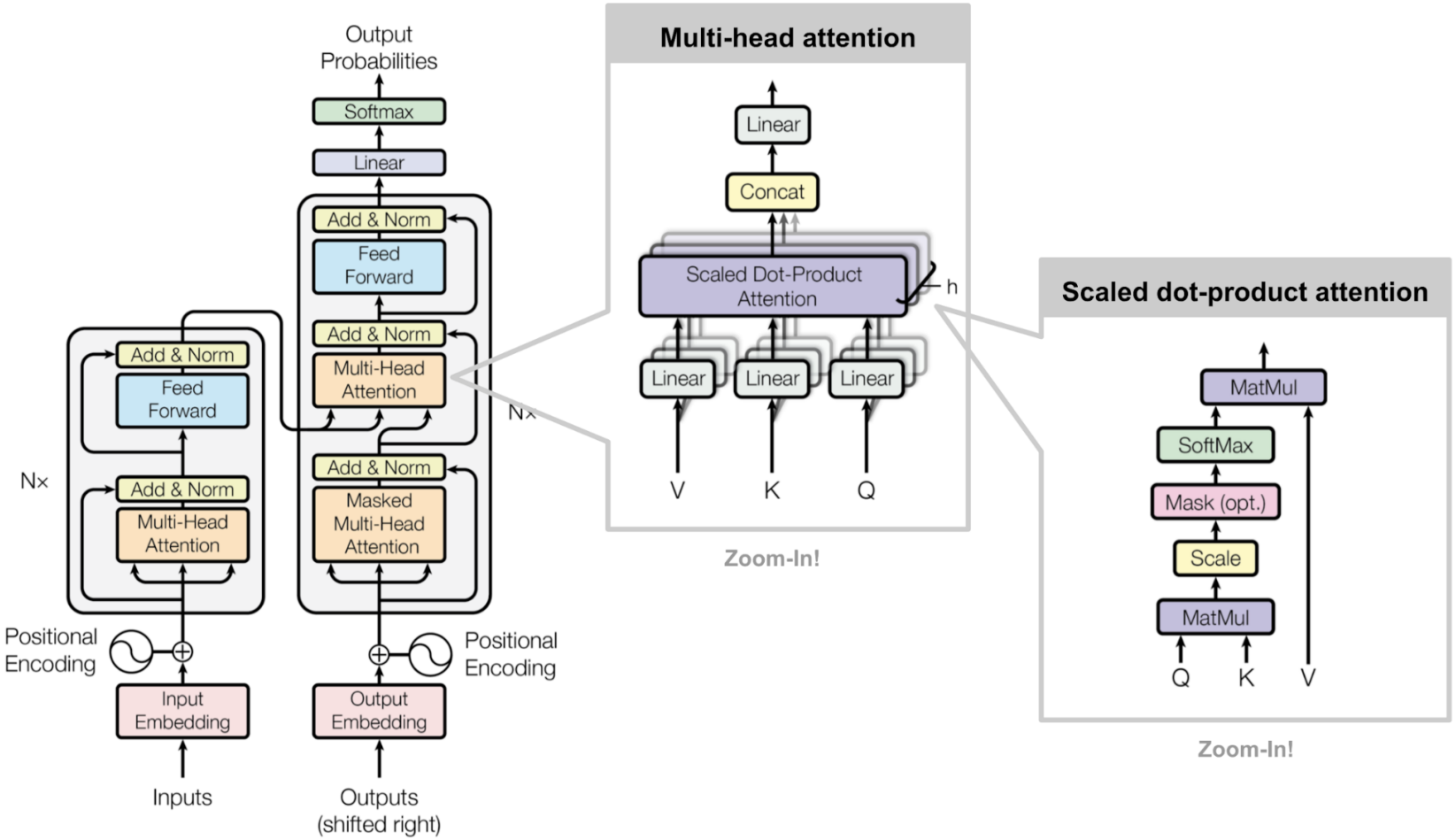




K, V: (encoded) state / memory / context, Q: embedding (why?)

- Input/Output Embedding ✓
- Positional Encoding
- Multi-Head Attention ✓; $\mathrm{MHA}(X, X, X)$
- Add & Norm
- Feed Forward
- Decoder
  - shifted right
  - Masked Multi-Head Attention
  - Cross Attention ✓; $\mathrm{MHA}(Y, M, M)$

Transformer [VSP+17]



Positional Encoding

$$h_t^\ell = \phi(W^\ell[h_t^{\ell-1}; h_{t-1}^\ell]) \qquad \text{(RNN)}$$
$$h^\ell = A(h^{\ell-1})h^{\ell-1} \qquad \text{(Attention)}$$

Since attention is permutation invariant (too weak inductive bias), it ignores the input word ordering. To overcome this, we can concatenate the word embeddings X with a positional embedding P, so that the model knows what order the words occur in. In practice, instead of concatenating X and P, we add them as follows:

$$\text{PE}(X) = X + P$$

Add & Norm

$$LN(X + F(X))$$ [18]

[18] BN is usually empirically less effective than LN in NLP taks.

Feed Forward

$$\text{d_model } (512) \rightarrow \text{dim_feedforward } (2048) \rightarrow \text{d_model } (512)$$

Masked MHA

We restrict attention to a subset of the dictionary by setting the attention score for the masked entries to a large negative number so that he corresponding softmax weights will be 0.

$$Q, K, V = YW^Q, YW^K, YW^V$$

$$[QK^\top]_{ij} = \begin{cases} q_i^\top k_j & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

$$A_{ij} = [\text{softmax}(QK^\top/\sqrt{d_k})]_{ij} = \begin{cases} \dots & \text{if } j \leq i \\ 0 & \text{if } j > i \end{cases}$$

The output (i) only depends on the past inputs $(j \leq i)$:

$$e_i^\top AV = A_{i*}V = \sum_{j \leq i} A_{ij} V_{j*}$$

Masked MHA

$$A_{ij} = [\text{softmax}(QK^\top/\sqrt{d_k})]_{ij} = \begin{cases} \dots & \text{if } j \leq i \\ 0 & \text{if } j > i \end{cases}$$

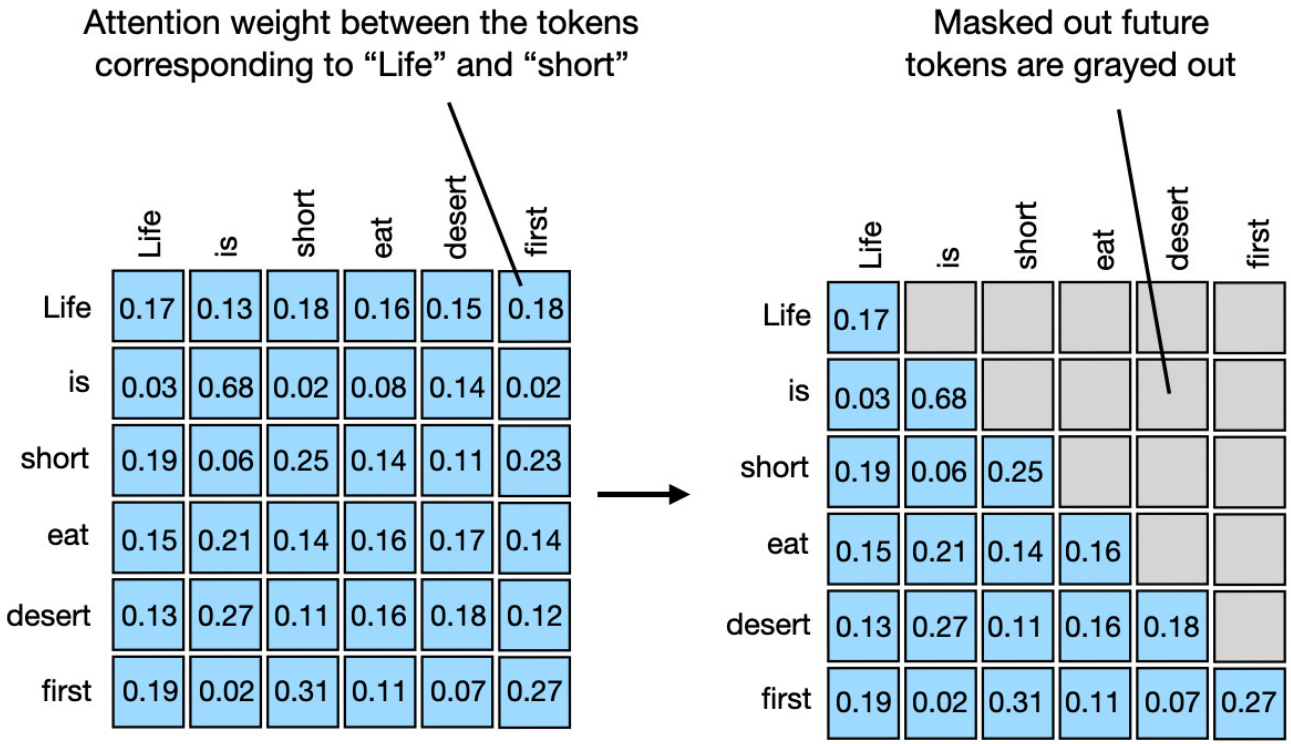
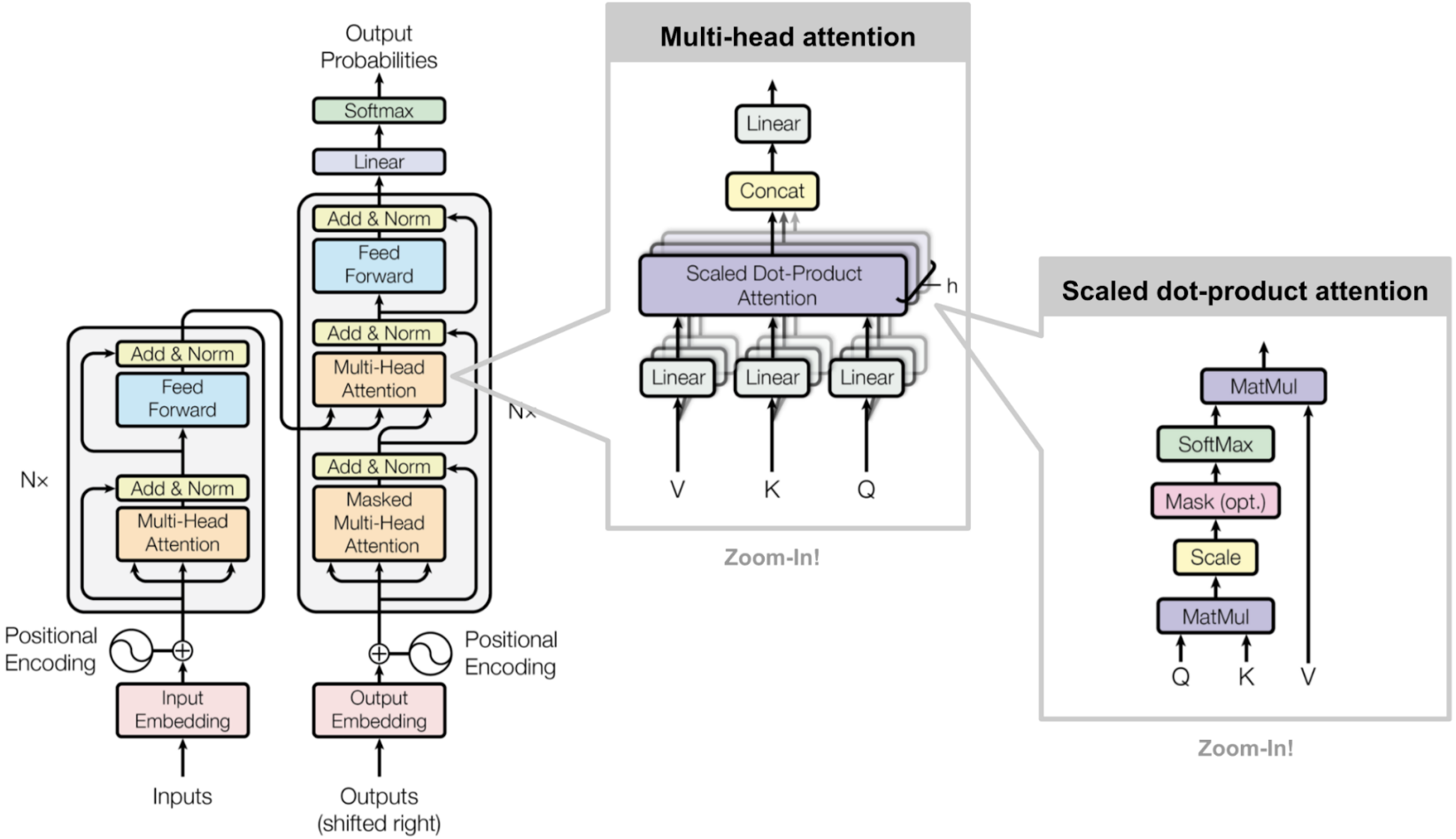
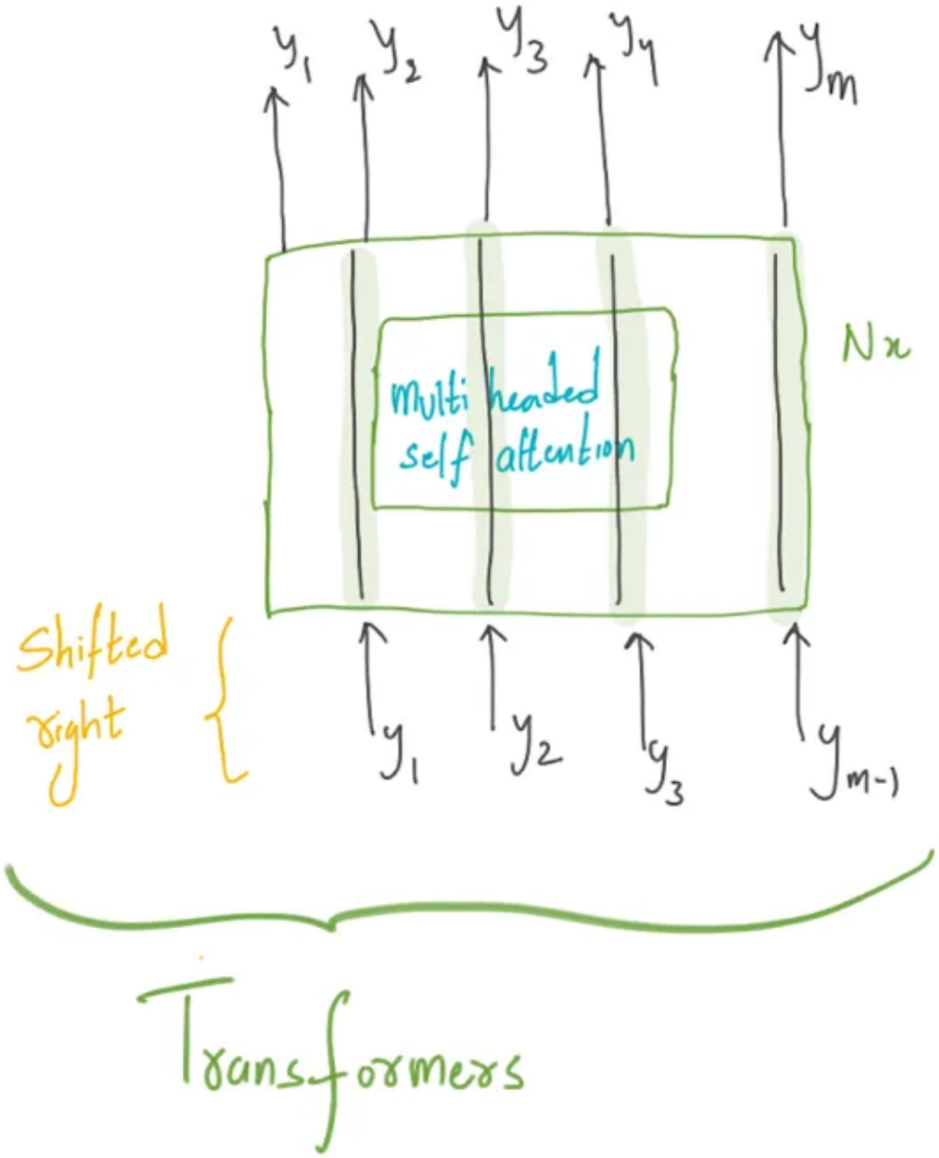


Figure: understanding-and-coding-self-attention

Transformer [VSP+17]



Shifted Right



- Input/Output Embedding ✓
- Positional Encoding ✓; $\mathrm{PE}(X) = X + P$
- Multi-Head Attention ✓; $\mathrm{MHA}(X, X, X)$
- Add & Norm ✓; $LN(X + F(X))$
- Feed Forward ✓; $\phi(XW)W'$
- Decoder
  - shifted right ✓; $y_{1:t-1} \mapsto y_t$
  - Masked Multi-Head Attention ✓; lower triangle A
  - Cross Attention ✓; $\mathrm{MHA}(Y, M, M)$

Pytorch Implementation

```python
class Transformer(Module):
    def __init__(self, d_model: int = 512, nhead: int = 8, num_encoder_layers: int = 6,
      num_decoder_layers: int = 6, dim_feedforward: int = 2048, dropout: float = 0.1, ...) ->
      None:
        ...
        encoder_layer = TransformerEncoderLayer(d_model, nhead, dim_feedforward, ...)
        encoder_norm = LayerNorm(d_model, ...)
        self.encoder = TransformerEncoder(encoder_layer, num_encoder_layers, encoder_norm)
        self.decoder = TransformerDecoder(decoder_layer, num_decoder_layers, decoder_norm)
        ...

    def forward(self, src: Tensor, tgt: Tensor, ...) -> Tensor:
        ...
        memory = self.encoder(src, ...)
        output = self.decoder(tgt, memory, ...)
        return output

class TransformerEncoder(Module):
    def __init__(self, encoder_layer, num_layers, norm, ...)
        ...
        self.layers = _get_clones(encoder_layer, num_layers)
        self.num_layers = num_layers
        self.norm = norm
        ...

    def forward(self, src: Tensor, ...) -> Tensor:
        ...
        output = src
        ...
        for mod in self.layers:
            output = mod(output, ...)

        if self.norm is not None:
            output = self.norm(output)

        return output
```

```python
class TransformerDecoder(Module):

    def forward(self, src: Tensor, ...) -> Tensor:
        ...
        output = tgt
        ...
        for mod in self.layers:
            output = mod(output, memory, ...)

        if self.norm is not None:
            output = self.norm(output)

        return output

class TransformerEncoderLayer(Module):
    def __init__(self, ...) -> None:
        self.self_attn = MultiheadAttention(d_model, nhead, ...)

        self.dropout1 = Dropout(dropout)
        self.norm1 = LayerNorm(d_model, ...)

        self.linear1 = Linear(d_model, dim_feedforward, ...)
        ## self.activation = F.relu
        self.dropout = Dropout(dropout)
        self.linear2 = Linear(dim_feedforward, d_model, ...)
        self.dropout2 = Dropout(dropout)
        self.norm2 = LayerNorm(d_model, ...)

    def forward(self, src: Tensor, ,,,) -> Tensor:
        ...
        x = src
        x = self.norm1(x + self._sa_block(x, ...) ## Add & Norm <- Self-Attention
        x = self.norm2(x + self._ff_block(x)) ## Add & Norm <- Fead Forward

        return x

    def _sa_block(self, x: Tensor, ...) -> Tensor:
        x = self.self_attn(x, x, x, ...)[0]
        return self.dropout1(x)

    def _ff_block(self, x: Tensor) -> Tensor:
        x = self.linear2(self.dropout(self.activation(self.linear1(x))))
        return self.dropout2(x)
```

```python
class TransformerDecoderLayer(Module):
    def __init__(self, ...) -> None:
        self.self_attn = MultiheadAttention(d_model, nhead, ...)
        self.multihead_attn = MultiheadAttention(d_model, nhead, ...)

        self.dropout1 = Dropout(dropout)
        self.norm1 = LayerNorm(d_model, ...)

        self.dropout2 = Dropout(dropout)
        self.norm2 = LayerNorm(d_model, ...)

        self.linear1 = Linear(d_model, dim_feedforward, ...)
        ## self.activation = F.relu
        self.dropout = Dropout(dropout)
        self.linear2 = Linear(dim_feedforward, d_model, ...)
        self.dropout3 = Dropout(dropout)
        self.norm3 = LayerNorm(d_model, ...)

    def forward(self, tgt: Tensor, memory: Tensor, , ,,,) -> Tensor:
        ...
        x = tgt
        x = self.norm1(x + self._sa_block(x, ...)) ## Add & Norm <- Self-Attention
        x = self.norm2(x + self._mha_block(x, memory, ...)) ## Add & Norm <- MHA
        x = self.norm3(x + self._ff_block(x)) ## Add & Norm <- Feed Forward

        return x

    def _mha_block(self, x: Tensor, mem: Tensor, ...) -> Tensor:
        x = self.multihead_attn(x, mem, mem, ...)[0] ## Q, K, V
        return self.dropout2(x)

    def _ff_block(self, x: Tensor) -> Tensor:
        x = self.linear2(self.dropout(self.activation(self.linear1(x))))
        return self.dropout3(x)
```

multi_head_attention_forward